LAUNCH CYAN

DevOps & Cloud [SWE] — Class 21 Notes

LECTURE TITLE	The Final DevOps Capstone Project — TaskBoard Walkthrough, the 10-Module Grading Rubric, a Live Docker Compose Run & the End-Term Exam Pattern *(the header says "Final Devops Project & Trouble Shooting_2" — see the note below)*
DATE/TIME	4 October 2026 (Sunday), 12:15 PM
DURATION	120 minutes
INSTRUCTOR	Nensi Ravaliya
BIG IDEA	One small three-tier Python app — frontend, backend, database — carried through **every** tool of the course (tests → Git → Docker → Compose → CI/CD → Trivy → Terraform → Kubernetes/ Helm → Prometheus/Grafana) is your final project, and it's worth **100 points / 30% of the end term**.
SOURCES	class transcript + her official folder for this session, `Nency-Ravaliya/devops-heros` → `session21-python` (`README.md`, `GRADING.md`, `backend/`, `frontend/`, `docker-compose.yml`, `.github/workflows/ci-cd.yml`, `terraform/`, `helm/`, `k8s/`, `monitoring/`, `scripts/`, `troubleshooting/`) — your copy is in `session21-python/` in this folder. The Docker-Hub-plus-kind pipeline she *described* is cross-checked against her `session-17-devsecops/demo/.github/workflows/devsecops.yml`. Monitoring background from `session20-monitoring-observability-gitops` (no transcript for that one).

WHAT'S INSIDE

0 ⚡ The 60-Second Version
1 🗓️ Logistics & Warm-Up — The Last Class
2 🖼️ What You're Building — TaskBoard, the Reference Project
3 📋 The 10 Graded Modules — The Full Rubric
4 🧪 M2 — Tests Come First
5 🐙 M3 — Git & GitHub
6 🐋 M4 — Dockerfiles & Docker Compose
7 🚀 M5 — The CI/CD Pipeline
8 🛡️ M6 — Trivy: The Security Gate
9 🏗️ M7 — Terraform
10 ☸️ M8 — Kubernetes & Helm: Two Folders, Two Jobs
11 📈 M9 — Prometheus & Grafana
12 🎤 M10 — README, PPT & the Bonus Video
13 🏋️ The Live Demo — `docker compose up`, a Crash, and the Fix
14 🏋️ The Full Practice Lab — Today's Homework, Start to Finish
15 🔧 Troubleshooting Lab — From Her Repo (Not Demoed)
16 🎓 The End-Term Exam Pattern
17 📅 Deadlines, Homework & the Four Things Left
18 ❓ Q&A Round-Up
19 📝 Cheat-Sheet
20 🗒️ Notes on the Transcript Itself

WATCH OUT

Read this first — the "Troubleshooting 2" half of the title didn't happen.

The transcript header says **"Final Devops Project & Trouble Shooting_2"**. What was actually taught:

1. the final-project brief — what to build, the **10 graded modules**, what to submit · **2.** a walk through her reference project, **TaskBoard** · **3.** how to set up the **Docker Hub token** and **GitHub secret** the pipeline needs · **4.** a **live** `docker compose up`, including one real failure (the backend died because Postgres wasn't ready) · **5.** the **end-term exam pattern** (MCQ + lab) · **6.** deadlines, homework and submission logistics.

There was **no Kubernetes troubleshooting session**. The only "troubleshooting" was the backend crash in §13, fixed by reading `docker logs` and re-running. Her repo *does* ship two broken manifests for practice (`troubleshooting/`), so §15 turns them into a lab — clearly marked as from the repo, not from the audio.

NOTE

Numbering. This is her **"session 21"** and your `Class-21` — for once they match. The class before was Class 20 (Cloud & Terraform VPC). Class 15 was "troubleshooting #1". **Section A** had this same briefing one day later, plus a GitOps session; this recording is **Section B's last class**.

0 ⚡ The 60-Second Version

1. **The final project = one Python three-tier app, run through every tool of the course.** Frontend + FastAPI backend + Postgres → tests → GitHub → Dockerfiles + Compose → GitHub Actions → Trivy → Terraform → Kubernetes + Helm → Prometheus + Grafana → README + PPT.
2. **100 points, 10 modules, 30% of the end term.** The heavy ones are **CI/CD (15)**, **Terraform (15)** and **Kubernetes + Helm (15)**. A skipped module scores zero.
3. **Your own domain, her architecture.** Copy the *DevOps layer* of TaskBoard; build your *own app* (CRM, clinic, inventory, helpdesk…). A straight clone of TaskBoard scores **zero**. **Python only** — no Java.
4. **The pipeline has five stages:** test → Docker build → Trivy scan → push to registry → deploy to a **kind** test cluster. Image tags = the **Git commit SHA**, never `latest`.
5. **Two Kubernetes folders, two jobs.** `k8s/` (just `deployment.yaml` + `service.yaml`) for the CI kind cluster; `helm/` for the "real" deploy (Minikube/EKS) with upgrade and rollback.
6. **Terraform: write it, don't necessarily apply it.** `main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`. No `provider.tf` unless you're multi-cloud.
7. **Two documents, two audiences.** `README.md` = technical "how to run it". The **PPT** = a non-technical person should understand *what* and *why*. A **≤2-minute video** earns **+5 bonus points**.
8. **Deadlines:** 21 homework questions → **Wednesday 7 Oct, 11:59 PM**. Final project → **Sunday 11 Oct**. End-term exam (MCQ + lab) → after **17–20 Oct**.

1 🗒️ Logistics & Warm-Up — The Last Class

Opening. "*Not audible? Really? Am I audible?*" — the usual mic check, then the headline:

> SAID IN CLASS
>
> "*Today we have our last session for batch B. For batch A, tomorrow we'll have one session — GitOps — and the same final DevOps project.*"

Who was there. She couldn't find the participant count at first ("*how many people are there inside this classroom? I don't know from where I can check*") — then: **135** live. Later, a poll counted **156** in the batch.

The plan she laid out for the two hours:

```
Overview: what to submit + grading components
        |
        ▼
Walk the reference repo (devops-heros → session21-python)
        |
        ▼
Create YOUR GitHub repo (e.g. "devsecops-python"), copy the code in
        |
        ▼
Run it:  manually  →  Docker  →  Docker Compose  →  Kubernetes  →  CI/CD
```

> SAID IN CLASS
>
> "*After today's session, you need to create the same type of application with a few features that we added inside our* `GRADING.md` *file and* `README.md` *file.*"

Getting the code. Her `devops-heros` repo was updated — "*open this link and then do the* `git pull`" — and the project lives in `session21-python`. She'd also built the same project in a separate repo of her own and already run its CI/CD pipeline several times before class.

Her availability, stated plainly:

When	Where she is
Tomorrow (Mon 5 Oct)	**On campus**, both sections — bring every doubt: DevOps, Kubernetes, monitoring, labs, deadlines
After that	**Not available** until the exam
During the exam window (after ~20 Oct)	Online, only if something goes wrong in the exam

CLASS MOMENT

Class moments. - A student asked about something "for Docker networking" — "*Docker networking? I didn't get you.*" It turned out to be about the **lab questions on the dashboard** (§16). - Late in the Q&A she realised her screen share had paused — "*screen share is paused? Oh my God, sorry sorry guys*" — and re-shared to show the homework form. - Attendance requests poured in at the end ("*I need only one class to cross 75%*"). A new user named Risha joined late; another student (Shreesha) was asked to **come offline tomorrow** because attendance needs *output* — "*you need to create the VPC from last session… I need some output from you guys to add your attendance.*" - One student said the dashboard wasn't working so they'd watched via a friend's account → "*discuss that with the management and program team.*"

2 🗺️ What You're Building — TaskBoard, the Reference Project

PICTURE THIS

Picture this: a driving test route. The examiner doesn't care *which car* you bring — hatchback, SUV, whatever (that's your app domain). They care that you can do *every manoeuvre on the route*: start, signal, merge, reverse-park, stop (that's test → build → scan → push → deploy → monitor). TaskBoard is her car, driven round the route once so you can see every turn.

TaskBoard is "*a small but realistic SaaS-style project-management application*" (her README):

Layer	Tech
Frontend	**React + Vite**, served by **Nginx** in production
Backend	**FastAPI** (Python), **SQLAlchemy** ORM, **Alembic** migrations
Database	**PostgreSQL 16**
Tests	**Pytest**
Containers	Two Dockerfiles + one `docker-compose.yml`
CI/CD	**GitHub Actions**
Security	**Trivy** image scan
Infra	**Terraform** → AWS **VPC + EKS**
Orchestration	**Kubernetes** + **Helm** + Ingress + HPA
Observability	**Prometheus + Grafana**

The journey her README draws — from laptop to a monitored cluster:



SAID IN CLASS

The point, in her README: "*The point is not to teach isolated tools. The point is to show how a real application travels from a developer laptop to a monitored Kubernetes environment.*"

2.1 The repo layout you're expected to mirror

```
devsecops-python/              ← your new GitHub repo
├── frontend/                  # React app + its own Dockerfile + .dockerignore + nginx.conf
├── backend/                   # FastAPI app + its own Dockerfile + .dockerignore
│   ├── app/                   #   main.py, models.py, schemas.py, db.py, config.py
│   ├── tests/                 #   pytest tests
│   └── alembic/               #   DB migrations
├── docker-compose.yml         # ROOT: all three services
├── .github/workflows/ci-cd.yml
├── terraform/                 # main.tf, variables.tf, outputs.tf, versions.tf
├── k8s/                       # deployment.yaml + service.yaml (for the CI kind cluster)
├── helm/taskboard/            # the "real" deployment chart
├── monitoring/                # Prometheus/Grafana values
├── scripts/                   # optional test helpers
├── docs/                      # optional: extra .md docs
├── README.md                  # technical doc + screenshots
└── <your-presentation>.pptx   # the final PPT, in the root
```

GOOD TO KNOW

No Dockerfile in the root. Each microservice keeps its own Dockerfile in its own folder. The root holds only the **Compose** file that builds them both: "*In your root folder you don't need to mention your Dockerfile, but what we need is your Docker Compose file.*"

2.2 The backend API

```
GET    /                  → {"service": "TaskBoard API", "version": "1.0.0", "docs": "/docs"}
GET    /health            → {"status": "UP"}            ← liveness
GET    /ready             → {"status": "READY"}         ← readiness (touches the DB)
GET    /metrics           → Prometheus text format
GET    /docs              → Swagger UI (FastAPI gives it for free)

GET    /api/tasks
GET    /api/tasks/{id}
POST   /api/tasks
PUT    /api/tasks/{id}
DELETE /api/tasks/{id}
GET    /api/tasks/stats   → {total, todo, inProgress, done}
```

WHY IT WORKS

Why three "am I OK?" endpoints? They answer three different questions.

Endpoint	Question it answers	Who asks
`/health`	"*Is the process alive?*" — cheap, touches nothing	Kubernetes **liveness** probe
`/ready`	"*Can I serve traffic right now?*" — runs a real DB query	Kubernetes **readiness** probe
`/metrics`	"*What's been happening?*" — counters, latencies	**Prometheus**

Her README: "*A container can be alive while its application is unhealthy.*" A backend whose DB is down is *alive* (`/health` = UP) but not *ready* (`/ready` errors) — so Kubernetes stops sending it traffic without killing it.

2.3 The frontend

A dark SaaS dashboard: sidebar, KPI cards (**Total / To do / In progress / Completed**), a task table with **status filters** and **priority badges**, a **Recent activity** feed, a **CI → Build → Scan → Deploy** pipeline strip, and a **create-task modal**. Clicking ↻ on a row moves a task **TODO → IN PROGRESS → DONE → TODO** (a `PUT`).

The browser calls `/api/tasks` and `/api/tasks/stats` — relative paths. It never knows the backend's hostname: **Nginx** (in Compose) or the **Ingress** (in Kubernetes) forwards `/api/…` to the backend.

CLASS MOMENT

"Good morning, Nensi 👋". The greeting, the profile card and the default assignee are **hard-coded** to her name — "*you will see 'Good morning' and maybe 'Nancy' because static name I added.*" Change it to yours; it's the first thing a grader sees.

---

3 📋 The 10 Graded Modules — The Full Rubric

HOMEWORK

> **Final Project — 100 points, 30% of your end-term grade.** Deadline: **Sunday 11 October 2026**. The rubric below is her `GRADING.md`, verbatim in substance; her spoken additions are marked 🎤.

Module	Topic	Points
M1	Application — Frontend + Backend + Database	10
M2	Testing — Pytest + code quality	10
M3	Git and GitHub	5
M4	Docker — Dockerfiles + Compose	10
M5	CI/CD — GitHub Actions pipeline	**15**
M6	DevSecOps — Trivy scan	5
M7	Terraform — AWS IaC	**15**
M8	Kubernetes + Helm	**15**
M9	Observability — Prometheus + Grafana	10
M10	Final presentation + documentation	5
	Total	**100**
	Bonus: a ≤2-minute project video 🎤	**+5**

> WATCH OUT
>
> **"A student who skips a module receives zero for that section — partial credit is awarded only within a module."** Every module needs *something*.

3.1 M1 — Application (10)

Criterion	Pts
FastAPI backend starts and answers `/health`	2
≥ 4 REST endpoints — **GET, POST, PUT, DELETE**	3
PostgreSQL, ≥ 1 table managed by **Alembic** migrations	2
React/HTML frontend renders and **calls the API**	2
Responsive, usable UI (not a blank page)	1

Submit: `backend/app/`, `backend/alembic/versions/`, `requirements.txt`, `frontend/`, `docker-compose.yml`, a screenshot/recording of the app in the browser.

🎤 **Her spoken additions:** - **The substitution rule.** *"If you are not using any one of it, then you need to implement some other feature and just mention: 'I did not implement this particular grading component, but I added a new feature.' For that new feature you will get the same points."* - **A real UI, not a toy.** *"I need actual UI — not directly a Claude UI that you are going to create… not a static application where I'm not getting any kind of output from your backend or database."* (She said "*responsible*"; she means **responsive**.) - **Structure matters:** `app.py` (or an `app/` package), `requirements.txt`,

`requirements-dev.txt` . - **Database choice is yours** — "*Postgres, MongoDB, Oracle DB, any kind of database you are okay with.*" (The rubric names Postgres + Alembic; if you switch, use the substitution rule and say so.)

3.2 M2 — Testing (10)

Criterion	Pts
`pytest` runs without errors	3
≥ **5 test cases** covering ≥ **3 endpoints**	4
Tests use a **test DB or mock**, not production	2
`pytest.ini` or `conftest.py` present and configured	1

Submit: `backend/tests/` , `pytest.ini` / `conftest.py` , a screenshot of `pytest -v` all green. 🎤 "*Test cases are very, very important.*"

3.3 M3 — Git and GitHub (5)

Criterion	Pts
Public repo linked in the submission form	2
Meaningful commit messages (not "update", "fix", "test")	2
`.gitignore` excludes `.env` , `__pycache__` , `node_modules` , `.venv`	1

Submit: repo URL + a commit-history screenshot with **≥ 10 commits**. 🎤 "*Don't think you just need to create a GitHub repo, clone it and add all the components. What we need here is your `.gitignore` file, maybe `.env` file — these standard practices.*"

3.4 M4 — Docker (10)

Criterion	Pts
`backend/Dockerfile` builds	2
`frontend/Dockerfile` is **multi-stage** (Node build → Nginx runtime)	3
Images run as **non-root** users	2
`docker compose up --build` starts all **three** services	3

Submit: both Dockerfiles, `docker-compose.yml` , a screenshot of `docker compose up --build` , a browser screenshot of `http://localhost:3000` .

3.5 M5 — CI/CD (15)

Criterion	Pts
A workflow file in `.github/workflows/`	1
Triggers on **push to** `main`	1
`pytest` runs in the pipeline and **fails the build** on failure	3
Frontend is built in the pipeline	2
Docker images built for **both** frontend and backend	3
Images **pushed to a registry** (rubric says GHCR — see §7.5)	3
Image tags = **Git commit SHA**, not `latest`	2

Submit: the workflow YAML, a link to a **green** run, a registry screenshot showing SHA tags.

3.6 M6 — DevSecOps / Trivy (5)

Criterion	Pts
Trivy scans **both** images in CI	3
Pipeline fails on **HIGH/CRITICAL** CVEs	1
You can explain one CVE found (or a clean result)	1

Submit: the Trivy step, a screenshot of its output, and **2–3 sentences** on what it scanned and what the result means.

3.7 M7 — Terraform (15)

Criterion	Pts
`terraform/` with valid HCL	2
`terraform init` clean	1
`terraform plan` non-empty, no errors	2
VPC with ≥ 2 public subnets	3
EKS cluster with ≥ 1 node group	4
`terraform destroy` tears everything down	2
`terraform.tfvars.example` present (no real creds)	1

Submit (per `GRADING.md`): all `.tf` files, `plan` output, AWS-console screenshot of VPC + EKS, `destroy` screenshot.
But see the 🔍 in §9 — what she *said* in class is much lighter.

> **WATCH OUT**
>
> **Never commit AWS keys.** "*The grading team will reject submissions where secrets are found in the commit history.*" — and deleting them in a later commit doesn't remove them from history.

3.8 M8 — Kubernetes + Helm (15)

Criterion	Pts
`k8s/namespace.yaml` applies cleanly	1
Helm chart: `Chart.yaml`, `values.yaml`, templates	3
`helm upgrade --install` deploys without errors	3
Backend + frontend Deployments with **≥ 2 replicas** each	2
ClusterIP Services for backend and frontend	2
Ingress routes `/` → frontend, `/api` → backend	2
`kubectl get pods -n <ns>` — all **Running**	2

Submit: `k8s/`, `helm/`, screenshots of `kubectl get pods`, `kubectl get svc`, `helm list`, and the app through the Ingress host/LB IP.

3.9 M9 — Observability (10)

Criterion	Pts
`/metrics` returns Prometheus format	2
Prometheus installed and **scraping** the app	3
Grafana installed and reachable	2
≥ 1 Grafana panel with **live** app metrics	3

Submit: `monitoring/` values, `curl …/metrics` screenshot, Prometheus **Targets** page showing **UP**, a populated Grafana dashboard.

3.10 M10 — Presentation + Documentation (5)

Criterion	Pts
Root `README.md` explaining what the app does	2
Live demo: commit → pipeline runs → deployment updates	3

🎤 Plus a **PPT** in the repo root and, optionally, a video for **+5**. Full detail in §12.

3.11 Grading policy

Rule	Detail
Late submission	**–5 points per 24 hours** after the deadline (🎤 *"we will deduct points as per the modules"*)
Plagiarism	Copy of another student, **or a direct clone of TaskBoard without meaningful changes → zero for all modules**
Domain	Must be your own. The DevOps layer *may* follow TaskBoard's architecture
AWS cost	Yours. Run `terraform destroy`
Language	🎤 **Python only** — *"Don't create a Java application… it will be easy for grading, easy to see test cases."*
Genuine emergencies	🎤 Ping her; if genuine, the deadline can move — but not past Sunday

GOOD TO KNOW

Ideas for "your own domain" (her README's list): CRM · inventory management · appointment booking · helpdesk · e-commerce admin · clinic management · restaurant management · employee management · learning management system.

3.12 The submission checklist

```
Application      [ ] repo URL submitted        [ ] runs via docker compose up --build
                 [ ] ≥ 4 REST endpoints         [ ] Alembic migration present
Testing          [ ] pytest passes (screenshot) [ ] ≥ 5 test cases
Docker           [ ] backend Dockerfile builds  [ ] frontend multi-stage
                 [ ] non-root user in BOTH Dockerfiles
CI/CD            [ ] workflow present           [ ] runs on push to main
                 [ ] pytest in pipeline          [ ] images pushed with SHA tags
Security         [ ] Trivy in pipeline           [ ] no secrets committed
Terraform        [ ] plan output                 [ ] VPC + EKS screenshot   [ ] destroy output
Kubernetes+Helm  [ ] get pods (all Running)      [ ] helm list              [ ] app via Ingress
Observability    [ ] /metrics screenshot         [ ] Prometheus Targets UP  [ ] Grafana panel
Documentation    [ ] README.md                   [ ] presentation (or recording)
```

4 🧪 M2 — Tests Come First

PICTURE THIS

Picture this: the bouncer at the door. Your pipeline is a nightclub queue. Pytest is the bouncer at the very front. If your code can't pass, it never gets *inside* — no image is built, nothing is scanned, nothing is pushed, nothing is deployed.

```
Bad code
  ↓
pytest fails
  ↓
Pipeline stops
  ↓
No broken image is promoted          ← "the first quality gate" (her README)
```

🎤 "*Last time also, for our Python application, we added the test cases. Same thing we need to add over here.*" (That was the session-17 DevSecOps demo, which ran `pytest --cov=app`.)

How her tests work (`backend/tests/test_api.py`):

```python
import os
os.environ["DATABASE_URL"] = "sqlite:///./test.db"   # ← test DB, NOT Postgres (M2: 2 pts)

from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_health():
    assert client.get("/health").json() == {"status": "UP"}

def test_root():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json()["service"] == "TaskBoard API"

def test_create_task_validation():
    response = client.post("/api/tasks", json={"title": "Deploy application",
                                               "priority": "HIGH", "assignee": "Student"})
    assert response.status_code == 201
```

WHY IT WORKS

Why the env var is set *before* the import. `app/db.py` builds the SQLAlchemy engine **at import time** from `settings.database_url`, and `pydantic-settings` reads `DATABASE_URL` from the environment. Set it after `from app.main import app` and the engine has already pointed itself at Postgres. Order is everything here.

`backend/pytest.ini` is two lines — `[pytest]` and `pythonpath = .` — which is what lets `from app.main import app` work when you run `pytest` from `backend/`.

WATCH OUT

Reviewing the reference tests — two things to fix in YOUR project.

1. There are only 3 tests. The rubric wants **≥ 5 tests across ≥ 3 endpoints**. Copying hers as-is loses points.

2. On a clean checkout, `test_create_task_validation` fails. I ran it (fresh folder, no `test.db`):

```
E   sqlalchemy.exc.OperationalError: (sqlite3.OperationalError) no such table: tasks
1 failed, 2 passed
```

The table is created in an `@app.on_event("startup")` hook, and **Starlette's `TestClient` only fires startup events inside a `with` block.** `client = TestClient(app)` on its own never runs it. In CI (always a clean checkout) the **test stage would go red and stop the whole pipeline**.

The fix — a `conftest.py` that also earns M2's config point:

```python
# backend/tests/conftest.py
import os
os.environ["DATABASE_URL"] = "sqlite:///./test.db"

import pytest
from fastapi.testclient import TestClient
from app.main import app

@pytest.fixture(scope="session")
def client():
    with TestClient(app) as c:      # ← "with" runs startup → create_all
        yield c
```

Then each test takes `client` as an argument. Running it with that change: `3 passed`.

TRY IT YOURSELF

Try it yourself — five tests that cover the CRUD loop.

```python
def test_list_empty_or_list(client):
    assert client.get("/api/tasks").status_code == 200

def test_create_then_get(client):
    t = client.post("/api/tasks", json={"title": "Write tests"}).json()
    assert client.get(f"/api/tasks/{t['id']}").json()["title"] == "Write tests"

def test_update_status(client):
    t = client.post("/api/tasks", json={"title": "Ship"}).json()
    r = client.put(f"/api/tasks/{t['id']}", json={"status": "DONE"})
    assert r.json()["status"] == "DONE"

def test_delete_then_404(client):
    t = client.post("/api/tasks", json={"title": "Temp"}).json()
    assert client.delete(f"/api/tasks/{t['id']}").status_code == 204
    assert client.get(f"/api/tasks/{t['id']}").status_code == 404

def test_rejects_bad_priority(client):
    assert client.post("/api/tasks", json={"title": "x", "priority": "URGENT"}).status_code == 422
```

The last one checks **validation**: `schemas.py` only allows `LOW | MEDIUM | HIGH`, so FastAPI returns **422 Unprocessable Entity** on its own.

5 🐙 M3 — Git & GitHub

🎤 *"Right now you need to create one GitHub repo — let's suppose the name you can give: `devsecops-python`. Create the GitHub repo, and inside it put whatever we have inside our `session21-python`."* She also called this module *"I guess very, very easy for you guys."*

Her README's sequence:

```
git init
git add .
git commit -m "initial TaskBoard application"
git branch -M main
git remote add origin <YOUR_GITHUB_REPO>
git push -u origin main
```

Term	One-liner (her README)
Git	version control
GitHub	remote collaboration / source platform
commit	immutable project checkpoint
branch	isolated line of development
pull request	controlled change review

LOCK THIS IN

Commit messages are graded (2 pts). `"update"`, `"fix"`, `"test"` score zero. Write what changed and why: `"Add PUT /api/tasks/{id} with partial updates"`, `"Pin trivy-action to 0.30.0"`. You also need **≥ 10 commits** — so commit as you build, module by module, not all at once at the end.

WATCH OUT

Her root `.gitignore` would cost you the M3 point. It contains `.idea/ .vscode/ node_modules/ dist/ target/ *.log .env .terraform/ *.tfstate*` — but **not `__pycache__/` and not `.venv/`**, both of which the rubric names. Proof it matters: your copy of `session21-python` already carries `backend/app/__pycache__/*.pyc` and `backend/tests/__pycache__/*.pyc`. Add:

```
__pycache__/
*.pyc
.venv/
.pytest_cache/
backend/test.db
```

(`target/` is a Java/Maven leftover — harmless, but a sign it was adapted from a Java project.)

GOOD TO KNOW

> `.env` **vs** `.env.example`**.** Her backend ships `.env.example` with a *placeholder* `DATABASE_URL`. That's the pattern: commit the **example**, ignore the **real** `.env`. Same idea as M7's `terraform.tfvars.example`.

---

6 🐳 M4 — Dockerfiles & Docker Compose

6.1 Two Dockerfiles, two folders

> **PICTURE THIS**
>
> **Picture this: two kitchens, one restaurant.** The bakery (frontend) and the grill (backend) each have their own recipe card in their own room. The restaurant manager (Compose) sits at the front with one sheet that says "*bakery is in room `./frontend`, grill is in room `./backend`, the fridge is a Postgres image off the shelf.*"

🎤 "*In the backend folder I have my Dockerfile… same, I have my frontend folder, inside it my own Dockerfile. And each time you can create a `.dockerignore` — `node_modules` you need to ignore, `.git` you need to ignore.*" The **database needs no Dockerfile** — "*for database, directly using the database*" image (`postgres:16-alpine`).

Backend Dockerfile:

```
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt && useradd --create-home --uid 10001 appuser
COPY alembic.ini ./
COPY alembic ./alembic
COPY app ./app
USER 10001
EXPOSE 8000
CMD ["sh", "-c", "alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port 8000"]
```

Line	Why it's there
`python:3.12-slim`	Small base — fewer packages, fewer CVEs for Trivy to find
`PYTHONDONTWRITEBYTECODE=1`	No `.pyc` clutter in the image
`PYTHONUNBUFFERED=1`	`print` /logs reach `docker logs` immediately, not in chunks
`COPY requirements.txt` **before** `COPY app`	Layer caching — dependencies only reinstall when `requirements.txt` changes
`useradd … --uid 10001` + `USER 10001`	**Non-root** (M4: 2 pts)
`--host 0.0.0.0`	Listen on every interface, so the port mapping can reach it (Docker's `0.0.0.0` rule again, Class 6)
`alembic upgrade head && uvicorn …`	Migrate the DB schema **first**, then start the API. If the migration fails, the container exits — which matters in §13

Frontend Dockerfile — multi-stage:

```
FROM node:22-alpine AS build          # stage 1: has npm, node_modules, Vite…
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build                     # → /app/dist (plain HTML/JS/CSS)

FROM nginx:1.27-alpine                # stage 2: only Nginx + the built files
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

PICTURE THIS

Multi-stage = build in the workshop, ship only the furniture. Stage 1 is the workshop full of saws and sawdust (Node, 300 MB of `node_modules`). Stage 2 is the delivery van carrying just the finished table (`dist/`). The saws never leave the workshop. Result: a tiny runtime image with nothing to exploit.

WATCH OUT

The reference frontend image runs as root — that's the M4 "non-root" points. The official `nginx` image starts its master process as **root**. The checklist says "*Non-root user in both Dockerfiles.*" One simple fix:

```
FROM nginxinc/nginx-unprivileged:1.27-alpine   # runs as uid 101
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 8080
```

…and change `listen 80;` → `listen 8080;` in `nginx.conf` and `"3000:80"` → `"3000:8080"` in Compose. (Non-root processes can't bind ports below 1024.)

Nginx is also the API router:

```
location /      { try_files $uri /index.html; }        # React app (SPA fallback)
location /api/  { proxy_pass http://backend:8000; … }   # → backend container
location /health { proxy_pass http://backend:8000/health; }
```

`backend` here is the **Compose service name**, resolved by Docker's built-in DNS on the Compose network (Class 8).

6.2 The Compose file

```yaml
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: taskboard
      POSTGRES_USER: taskboard
      POSTGRES_PASSWORD: taskboard
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql/data     # ← data survives the container

  backend:
    build: ./backend                               # ← builds backend/Dockerfile
    environment:
      DATABASE_URL: postgresql+psycopg://taskboard:taskboard@postgres:5432/taskboard
    depends_on:
      - postgres
    ports:
      - "8000:8000"

  frontend:
    build: ./frontend                              # ← builds frontend/Dockerfile
    depends_on:
      - backend
    ports:
      - "3000:80"

volumes:
  postgres-data:                                   # ← declared once, at the end
```

What she stressed, in order:

1. `build: ./backend` / `build: ./frontend` — *"it will build your Docker image from this backend folder; same for your frontend."*
2. **A volume for the database** — *"one of the best practices we learned: every time I create my database, I attach a volume. Suppose my database container is deleted — then your data will be lost."*
3. `depends_on` — *"Backend depends on Postgres. Once your Postgres container is up and running, only then the backend container will be created… So once your frontend is running, that means your backend and database always run."* (🔍 §13 — this is the half-truth that bit her live.)
4. **Declare the volume at the bottom** — *"whatever volume we created over here, you need to mention at the end."*
5. **Networks — optional extra credit.** *"If you want to try something new, you want to leverage this project."*

GOOD TO KNOW

The four Docker network drivers, as she listed them (Class 8 again):

Driver	Use here?
bridge	✅ the sensible choice — one private network for all three services
host	possible, but you lose port isolation (and it behaves oddly on macOS Docker Desktop)
overlay	❌ needs **2–3 Docker hosts** (Swarm) — "*I know you have your laptops only*"
none	❌ "*None network you don't need to create right now*" — the containers couldn't talk

```yaml
services:
  backend:
    networks: [taskboard-net]
  # …same for postgres and frontend…
networks:
  taskboard-net:
    driver: bridge
```

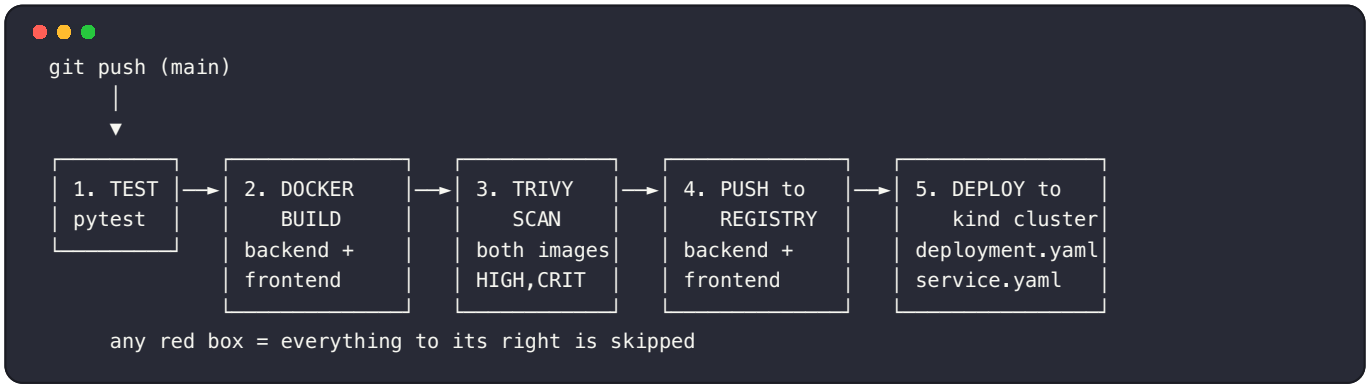
Compose already creates a default bridge network for you — an explicit one just makes it visible and lets you isolate, e.g. keep Postgres off the frontend's network.

---

7 🚀 M5 — The CI/CD Pipeline

7.1 The five stages she showed

She opened her own project repo → **Actions** and showed a stack of runs: "*testing is done, build is done, Trivy scan is also done, then pushing the image to Docker Hub, and deploying Kubernetes.*"



CLASS MOMENT

Her pipeline wasn't fully green either. "*See — that's fine, my one step is not done, but yeah, like this you need to create.*" Her deploy step had been pointed at an **EKS** cluster ("*I just added my EKS cluster, so that is the one thing I need to redo*"); for a kind cluster "*I just need to re-add this type and it will work.*"

🎤 **Two rules on the trigger:** - "*Make sure you add* `on: push: branches: main`. *Whenever you do the git commit, it will automatically run your CI/CD pipeline.*" - "*It's not only for the CI/CD pipeline — for GitOps concepts, we need that.*" (GitOps = Git is the source of truth; a push *is* a deployment request. Section A covers it in their extra session.)

🎤 **Security stage, reused:** "*In the CI/CD you need to mention some security tools. Last time we did Trivy, right? Same step you can copy from your CI/CD pipeline and put inside this new pipeline.*"

7.2 🏋️ Lab — creating a Docker Hub Personal Access Token

The push stage logs in to Docker Hub. You never put your password in the pipeline — you make a **token**.

DO THIS YOURSELF

Do this yourself — Docker Hub PAT.

1. Log in at **hub.docker.com**.
2. Click your **profile** → **Account settings**.
3. Open **Personal access tokens** → **Generate new token**.
4. **Description:** e.g. `github-actions-token` (she typed "*Docker Hub token*").
5. **Expiration: 30 days** — "*as a best practice.*"
6. **Access permissions: Read & Write**. "*Delete is not necessary — if you want to give then you can, but as a standard approach we will add repo read and write.*"
7. **Generate.** Docker Hub shows the token **once**, plus a ready-made login command:

```
docker login -u <your-dockerhub-username>
# Password:  <paste the token here, not your account password>
```

1. Copy the token somewhere safe *now* — you can't view it again.

PICTURE THIS

Why a token, not your password? A token is a **spare key cut for one job**. It can be limited (read/write but no delete), it expires on its own (30 days), and if it leaks you revoke *that key* without changing your password or locking yourself out.

7.3 🏋️ Lab — storing it as a GitHub Actions secret

DO THIS YOURSELF

Do this yourself — repository secret.

1. Your GitHub repo → **Settings**.
2. Left sidebar → **Secrets and variables** → **Actions**.
3. **New repository secret**.
4. **Name:** `DOCKERHUB_TOKEN` (she showed hers as "*Docker Hub token*"). **Secret:** paste the PAT.
5. **Add secret.** It now appears in the list, value hidden forever.
6. Reference it in the workflow as `${{ secrets.DOCKERHUB_TOKEN }}` — "*the same secret that I'm using inside my GitHub Actions pipeline.*"

```yaml
- name: Login to Docker Hub
  uses: docker/login-action@v3
  with:
    username: <your-dockerhub-username>
    password: ${{ secrets.DOCKERHUB_TOKEN }}
```

WATCH OUT

GitHub masks secrets in logs (`***`) — but only the exact string. Don't `echo` it, base64 it or split it in a `run:` step.

7.4 What each stage does, in her words

- **Test** — `pytest`. Nothing else runs if it fails.
- **Build** — "*It will build your backend image, it will build your frontend image.*"
- **Scan** — "*Install Trivy, then scan your backend image and scan your frontend image.*" Installing Trivy via `apt` commands is the simple path; "*if you don't want to install Trivy manually by these commands, you can use the Trivy Docker image — that is a bit difficult, but you can try.*"
- **Tag** — "*We are putting the GitHub SHA value — a new random value for each Docker image.*"
- **Push** — "*Two images: one for your backend and one for your frontend.*"
- **Deploy** — "*Make sure you use the kind cluster, which is a testing cluster that we deployed in the last pipeline… these are my files to deploy my Kubernetes objects:* `deployment.yaml` *and* `service.yaml`."

WHY IT WORKS

Why the commit SHA as the tag? Her README puts it as `commit A → image A, commit B → image B`. A tag like `latest` is a sticky note that keeps moving — you can never tell *which* code is running. A SHA tag is a **receipt number**: from any running pod you can trace back to the exact commit, and **rolling back** means redeploying an older SHA. It's also why the rubric gives 2 points for it.

7.5 🔍 Her spoken pipeline vs the file in the repo

CLARIFIED

Clarified, since what she *described* and what `session21-python/.github/workflows/ci-cd.yml` *contains* are different pipelines.

	What she said / showed live	`ci-cd.yml` in `session21-python`
Registry	**Docker Hub**, `secrets.DOCKERHUB_TOKEN`	**GHCR** (`ghcr.io`), built-in `secrets.GITHUB_TOKEN`
Trivy	install via `apt` , run `trivy image`	`aquasecurity/trivy-action@0.30.0` , `exit-code: '1'`
Deploy target	**kind** cluster created inside the runner	a **real** cluster via `secrets.KUBE_CONFIG_DATA`
Deploy method	`kubectl apply -f k8s/deployment.yaml` + `service.yaml`	`helm upgrade --install … --set backend.tag=${{ github.sha }}`
Jobs	5 stages	3 jobs: `test` → `build-scan-push` → `deploy`

Which to build? The rubric (M5) literally says "*pushed to GitHub Container Registry (GHCR)*". The pipeline she ran and told you to copy pushes to Docker Hub. **The safe answer: push to a registry with SHA tags, and say which one in your README.** If you want the rubric's exact wording covered, GHCR needs **no extra secret at all** — `permissions: packages: write` plus `secrets.GITHUB_TOKEN` is enough. Likewise, the repo's `deploy` job needs a real cluster's kubeconfig that you don't have — so for *your* project, the kind-cluster deploy she described is the one that will actually go green.

The kind-cluster deploy stage, as in her session-17 pipeline (the one she said to reuse):

```yaml
deploy:
  needs: push
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  steps:
    - uses: actions/checkout@v4
    - name: Create k8s Kind Cluster (for testing deployment)
      uses: helm/kind-action@v1.10.0
    - name: Update deployment with new image tag
      run: sed -i "s|__IMAGE_TAG__|${{ github.sha }}|g" k8s/deployment.yaml
    - name: Deploy to Kubernetes
      run: |
        kubectl apply -f k8s/deployment.yaml
        kubectl apply -f k8s/service.yaml
    - name: Check rollout status
      run: kubectl rollout status deployment/<your-deployment> --timeout=60s
```

WATCH IT HAPPEN

Watch it happen — one `git push` , frame by frame.

```
Frame 1  You: git commit -m "Add assignee filter" && git push origin main
Frame 2  GitHub sees a push to main → matches `on: push: branches: [main]` → queues the workflow
Frame 3  Job "test": fresh Ubuntu VM → checkout → setup-python 3.12 → pip install → pytest
           ✅ 5 passed   (❌ would end the run here, red X on the commit)
Frame 4  Job "build" (needs: test): docker build backend:<sha>, frontend:<sha>
Frame 5  Job "scan": trivy image --severity HIGH,CRITICAL on both
           ✅ 0 HIGH/CRITICAL  (❌ exit-code 1 → stop; nothing vulnerable gets pushed)
Frame 6  Job "push": docker login with ${{ secrets.DOCKERHUB_TOKEN }} → docker push ×2
           Docker Hub now shows tags  a1b2c3d…  for both repos
Frame 7  Job "deploy": kind-action spins up a throwaway cluster INSIDE the runner
           → sed swaps __IMAGE_TAG__ for a1b2c3d… → kubectl apply → rollout status ✅
Frame 8  The runner VM is destroyed. The kind cluster dies with it. The images stay in the registry.
```

Frame 8 is why the kind deploy is a **test**, not a real deployment: it proves your manifests and images *work together*, then evaporates.

---

8 M6 — Trivy: The Security Gate

Both scans in the repo pipeline:

```yaml
- name: Scan backend
  uses: aquasecurity/trivy-action@0.30.0
  with:
    image-ref: ghcr.io/${{ github.repository_owner }}/taskboard-backend:${{ github.sha }}
    severity: HIGH,CRITICAL
    ignore-unfixed: true
    exit-code: '1'
```

Setting	Meaning
`severity: HIGH,CRITICAL`	Only report the serious ones
`exit-code: '1'`	**Fail the job** if any are found → the rubric's "fail on HIGH/CRITICAL" point
`ignore-unfixed: true`	Skip CVEs that have **no patch yet** — you can't act on them anyway

WATCH OUT

The session-17 pipeline you might copy does *not* fail. It runs `trivy image --severity HIGH,CRITICAL <image>` with **no** `--exit-code 1` — Trivy prints the CVEs and exits 0, so the pipeline sails on. Add `--exit-code 1` or you lose M6's "fail the pipeline" point.

PICTURE THIS

Trivy is one smoke detector, not the fire brigade. Her README: "*A security scanner is not a magic guarantee of security. It is one automated control in the pipeline.*" The layers, which she listed as things to understand:

Layer	Checks	Tool you met (Class 18)
SAST	your source code for bad patterns	CodeQL
Dependency scanning (SCA)	the libraries you import	`pip-audit`
Secret scanning	passwords/keys committed to Git	GitHub secret scanning, Gitleaks
Container scanning	OS packages + libs inside the image	**Trivy**
Runtime security	what the container *does* when running	(not covered)

LOCK THIS IN

The "explain one CVE" point (1 pt). Write 2–3 sentences like: "*Trivy scanned the OS packages and Python libraries in `taskboard-backend:<sha>`. It found 0 HIGH/CRITICAL fixable CVEs, because the slim base image ships few packages. A clean scan means no known, patched, high-severity vulnerability — not that the image is 'secure'.*"

---

9 🏗️ M7 — Terraform

🎤 "*Inside Terraform you just need to mention your `main.tf`, where you can mention all the resources. Second one, `outputs.tf` — your output variables. Then `variables.tf` — your variables. And `versions.tf` — your provider and version detail.*"

File	Holds
`main.tf`	the **resources** / modules (VPC, EKS)
`variables.tf`	inputs: `aws_region = ap-south-1`, `cluster_name = taskboard-eks`, `environment = dev`
`outputs.tf`	what to print after apply: `cluster_name`, `cluster_endpoint`, `vpc_id`
`versions.tf`	`required_version >= 1.7.0`, `hashicorp/aws ~> 5.0`, the `provider "aws"` block

GOOD TO KNOW

No `provider.tf`? 🎤 "*You don't need to create `provider.tf` as of now, because we're only deploying to AWS cloud. We don't have a multi-cloud platform. If you have multi-cloud, only then do you need a `provider.tf` with your multi-cloud environment.*" Terraform doesn't care about filenames — every `.tf` in the folder is read as one config — so putting the single `provider "aws"` block in `versions.tf` is purely a tidiness choice.

What her `main.tf` builds (two community modules):

```
AWS  ap-south-1 (Mumbai)
 └── VPC  taskboard-vpc   10.20.0.0/16
      ├── public  10.20.101.0/24  (ap-south-1a)
      ├── public  10.20.102.0/24  (ap-south-1b)
      ├── private 10.20.1.0/24    (ap-south-1a)  ─┐
      ├── private 10.20.2.0/24    (ap-south-1b)  ─┼── EKS taskboard-eks (k8s 1.31)
      └── 1 × NAT gateway (single_nat_gateway)    │     node group "main": t3.medium
                                                  │      min 2 · desired 2 · max 4
                                                  └── nodes live in PRIVATE subnets
```

PICTURE THIS

Why bother with Terraform? Her README: instead of clicking *AWS Console → VPC → Subnet → EKS → Nodes…*, "*we describe infrastructure in code and let Terraform reconcile the desired state.*" Kubernetes only manages workloads; **it can't create the VPC or the cluster it runs on** — that's Terraform's job.

CLARIFIED

Clarified — how much Terraform do you actually have to *run*?

Source	Says
🎤 In class	"*I know you don't have credentials… you can use the same Terraform files for your project. You don't need to do* `terraform apply` *or any kind of command — I will just see your Terraform infrastructure files. That is enough for your submission.*" And: "*No need to do apply command or anything, because you don't have credentials as of now.*"
`GRADING.md` (M7)	`init` (1) + `plan` (2) + VPC provisioned (3) + EKS provisioned (4) + `destroy` (2) + console screenshots

Her spoken instruction is the more recent and the one she'll grade by, given no one has AWS credentials. **But `terraform init`, `terraform fmt` and `terraform validate` need no AWS credentials at all** — run them and screenshot them; it's free evidence for 3+ points. (`plan` *does* need credentials, because it reads live AWS state.) She also mentioned providing credentials "*for today's deployment*" — if you get them, run `plan` / `apply`, screenshot, and `destroy` **the same day** (EKS + NAT gateway bill by the hour).

WATCH OUT

Her `.tf` files don't parse — fix them before you copy. Every block is written on **one line**:

```
module "vpc" { source = "terraform-aws-modules/vpc/aws" version = "5.8.1" name = "taskboard-vpc" … }
```

HCL only allows a one-line block if it contains **one** argument. I ran `terraform validate` on your copy:

```
Error: Invalid single-argument block definition
  on main.tf line 1, in module "vpc":
A single-line block definition must end with a closing brace immediately
after its single argument definition.
```

Same error for `module "eks"` and the `terraform { … }` block in `versions.tf`. (`variables.tf` and `outputs.tf` are fine — one argument each.) The fix is just line breaks:

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "5.8.1"

  name            = "taskboard-vpc"
  cidr            = "10.20.0.0/16"
  azs             = ["ap-south-1a", "ap-south-1b"]
  private_subnets = ["10.20.1.0/24", "10.20.2.0/24"]
  public_subnets  = ["10.20.101.0/24", "10.20.102.0/24"]

  enable_nat_gateway = true
  single_nat_gateway = true
}
```

Do the same for `module "eks"` and `versions.tf`, then `terraform fmt` will keep it tidy. Also add the **`terraform.tfvars.example`** the rubric asks for (1 pt) — it isn't in her folder:

```
# terraform.tfvars.example — copy to terraform.tfvars (gitignored) and edit
aws_region   = "ap-south-1"
cluster_name = "taskboard-eks"
environment  = "dev"
```

DO THIS YOURSELF

Do this yourself — Terraform with zero AWS credentials.

1. `cd terraform`
2. Fix the one-line blocks as above.
3. `terraform fmt -recursive` — rewrites spacing; no output means already tidy.
4. `terraform init` — downloads the AWS provider and the two modules into `.terraform/`. ✅ screenshot.
5. `terraform validate` → `Success! The configuration is valid.` ✅ screenshot.
6. *(Only with credentials)* `terraform plan` → expect ~50+ resources "to add".
7. *(Only with credentials)* `terraform apply` → then `aws eks update-kubeconfig --region ap-south-1 --name taskboard-eks`.
8. *(Same day!)* `terraform destroy`.

Her `terraform/.gitignore` already excludes `.terraform/`, `*.tfstate*` and `.terraform.lock.hcl`, so none of the downloaded junk or state is committed.

---

10 ☸ M8 — Kubernetes & Helm: Two Folders, Two Jobs

🎤 *"I created two types of deployment charts. One is regarding your Helm and the second one is regarding your k8s."*

PICTURE THIS

Picture this: a fire drill vs a real move. The `k8s/` folder is the **fire drill** — two plain files you run in the CI's throwaway kind cluster to prove "yes, it starts." The `helm/` folder is the **real house move** — packaged, versioned, with upgrade and rollback — for a cluster that will still exist tomorrow.

	`k8s/`	`helm/taskboard/`
Used by	the **CI pipeline** (kind)	you, by hand — **Minikube**, or **EKS/AKS** if you have access
Contents	`deployment.yaml` + `service.yaml` (+ `namespace.yaml`)	`Chart.yaml`, `values*.yaml`, `templates/`
Why	*"It will be hectic for you to use the kind cluster plus test your API plus all this Helm chart… very tough on a local/testing cluster"*	release, **upgrade, rollback**, per-environment values
If you HAVE EKS/AKS	*"you don't need to create this k8s folder"* — just use Helm	✅

🎤 *"I will give you the freedom just to create one k8s folder. Then you put `deployment.yaml` and `service.yaml`. That's how we test through the CI/CD."*

WATCH OUT

Her `k8s/` folder only has `namespace.yaml`. The `deployment.yaml` + `service.yaml` she described for the kind deploy aren't in `session21-python` — you write them. A minimal backend pair (the session-17 pattern, with the `__IMAGE_TAG__` placeholder the pipeline `sed`s):

```yaml
# k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata: {name: backend}
spec:
  replicas: 2
  selector: {matchLabels: {app: backend}}
  template:
    metadata: {labels: {app: backend}}
    spec:
      containers:
        - name: backend
          image: <dockerhub-user>/taskboard-backend:__IMAGE_TAG__
          ports: [{containerPort: 8000}]
          readinessProbe: {httpGet: {path: /health, port: 8000}}
---
# k8s/service.yaml
apiVersion: v1
kind: Service
metadata: {name: backend}
spec:
  selector: {app: backend}          # ← must match the pod labels (see §15)
  ports: [{port: 8000, targetPort: 8000}]
```

In a bare kind cluster there's no Postgres, so the backend's `alembic upgrade head` would fail — either add a Postgres Deployment to `k8s/`, or probe `/health` (no DB) and keep the CI deploy a smoke test. Name the Service `backend` — then the frontend's `nginx.conf` (`proxy_pass http://backend:8000`) works unchanged.

10.1 Inside her Helm chart

```
helm/taskboard/
├── Chart.yaml              name: taskboard · version 1.0.0
├── values.yaml             defaults: replicaCount 2, HPA on, ingress off
├── values-dev.yaml         replicaCount 1, ingress ON (taskboard.local), HPA off
├── values-prod.yaml        replicaCount 2, ingress ON (taskboard.example.com), HPA on
└── templates/
    ├── _helpers.tpl        "taskboard.fullname" = <release>-taskboard
    ├── backend-deployment.yaml   probes: readiness /ready · liveness /health
    ├── backend-service.yaml      port 8000, named "http"
    ├── frontend-deployment.yaml  probes on "/" port 80
    ├── frontend-service.yaml     port 80
    ├── postgres.yaml       Secret + Service + Deployment + 5Gi PVC
    ├── ingress.yaml        /api → backend · / → frontend   (if ingress.enabled)
    ├── hpa.yaml            2→6 pods at 60% CPU              (if hpa.enabled)
    └── servicemonitor.yaml scrape /metrics every 15s        (if monitoring on)
```

The five Helm words her README wants you to know: **chart** (the package) · **values** (the knobs) · **templates** (YAML with blanks) · **release** (one installed copy) · **upgrade / rollback** (move between revisions).

```
kubectl apply -f k8s/namespace.yaml
helm upgrade --install taskboard ./helm/taskboard -n taskboard --create-namespace
helm upgrade --install taskboard ./helm/taskboard -n taskboard -f helm/taskboard/values-dev.yaml
helm list -n taskboard
helm history taskboard -n taskboard
helm rollback taskboard 1 -n taskboard
kubectl get pods,svc,hpa -n taskboard
```

WHY IT WORKS

`upgrade --install` **is the idempotent one.** `install` fails if the release exists; `upgrade` fails if it doesn't. `upgrade --install` works either way — which is why every pipeline uses it.

GOOD TO KNOW

Ingress needs a controller. Her README: "*An Ingress resource is only configuration. An Ingress Controller must actually implement it.*" On Minikube: `minikube addons enable ingress`, then map `taskboard.local` to the Minikube IP in `/etc/hosts` (on macOS with the Docker driver, use `minikube tunnel` and `127.0.0.1` instead — Docker Desktop's VM again).

GOOD TO KNOW

HPA needs two things (her README): **resource requests** on the pods (`cpu: 100m` in `values.yaml`) and a **metrics provider** (`minikube addons enable metrics-server`). Without them, `kubectl get hpa` shows `<unknown>/60%` forever. And "*a normal health request may not create enough CPU pressure to demonstrate scaling.*"

WATCH OUT

Two naming bugs in her chart that will break the "real" deploy. I rendered it with `helm template taskboard ./helm/taskboard -f values-dev.yaml`:

Resource	Actually named	Something else expects
backend Service	`taskboard-taskboard-backend`, port **8000**	**Ingress** routes `/api` → `taskboard-backend`, port **8080** ❌
backend Service	`taskboard-taskboard-backend`	**frontend** `nginx.conf` proxies to `backend:8000` ❌

1. Ingress → `/api` goes to a Service that doesn't exist, on the wrong port → **503**. Fix in `ingress.yaml`: `name: {{ include "taskboard.fullname" . }}-backend`, `number: 8000`.

2. Frontend → Nginx resolves `proxy_pass` hosts **at startup**; there is no Service called `backend` in the namespace, so the frontend pod logs `host not found in upstream "backend"` and **CrashLoops**. Simplest fix: name the backend Service plainly `backend` (`metadata: name: backend` in `backend-service.yaml`) and point the Ingress at `backend:8000` too.

These are exactly the "Service name / port / selector doesn't match" faults from Class 15 — worth fixing *and* writing up in your README as "issues I debugged".

---

11 📈 M9 — Prometheus & Grafana

🎤 "*On top of it, you need to add observability tools and monitoring tools like Prometheus and Grafana.*" Section B had already done Prometheus + Grafana; "*for Section A, you will learn it tomorrow.*"

PICTURE THIS

Picture this: the fitness tracker and the app. `/metrics` is your **smartwatch** — it silently counts every request, every millisecond. **Prometheus** is the phone that **syncs** with the watch every 15 seconds and stores the history. **Grafana** is the **app with the pretty charts**. The watch never pushes; the phone pulls.

🎤 "*Whenever something happens within your application, Prometheus will fetch the data and then it will do the query, and the same query we are accessing through our Grafana dashboard.*"

How the pieces connect in her project:

```
FastAPI ── prometheus-fastapi-instrumentator ──► GET /metrics
                                                      ▲
                    ServiceMonitor (helm/templates)   │ every 15s, port "http"
                    label release: kube-prometheus-stack
                                                      │
                         Prometheus (kube-prometheus-stack) ── PromQL ──► Grafana panel
```

`Instrumentator().instrument(app).expose(app, endpoint="/metrics")` in `main.py` is the one line that creates the endpoint. Her `monitoring/prometheus-values.yaml` turns on Grafana and tells Prometheus to pick up **every** ServiceMonitor in **every** namespace:

```yaml
grafana:
  enabled: true
prometheus:
  prometheusSpec:
    serviceMonitorSelectorNilUsesHelmValues: false   # don't only watch Helm-labelled monitors
    serviceMonitorNamespaceSelector: {}              # {} = all namespaces
```

DO THIS YOURSELF

Do this yourself — monitoring on Minikube (commands filled in from the standard `kube-prometheus-stack` chart, which the ServiceMonitor's `release:` label points at):

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm upgrade --install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace -f monitoring/prometheus-values.yaml

kubectl port-forward -n monitoring svc/kube-prometheus-stack-prometheus 9090:9090
#   → http://localhost:9090/targets   — look for your backend, State = UP   (screenshot!)
kubectl port-forward -n monitoring svc/kube-prometheus-stack-grafana 3001:80
#   → http://localhost:3001  (admin / prom-operator by default)
```

A first Grafana panel query: `sum(rate(http_requests_total[1m])) by (handler)` — requests per second per endpoint.

GOOD TO KNOW

Without Kubernetes: her session-20 folder runs Prometheus + Grafana as plain **Docker Compose** with a `prometheus.yml` scrape config — a perfectly good route to M9 screenshots if your cluster is giving you grief. Point the scrape target at `backend:8000`.

Her load-test helper (`scripts/load-test.sh`) — *"for your testing purpose… not necessary to add, but for your submission you can take it as a reference"*:

```
URL="${URL:-http://taskboard.local/api/health}"
for i in $(seq 1 500); do curl -s "$URL" >/dev/null || true; done
```

WATCH OUT

Small bug: the backend has `/health`, not `/api/health`. Through the Ingress, `/api/health` reaches the backend as `/api/health` → **404** — 500 fast 404s won't push CPU or make interesting graphs. Run it as `URL=http://taskboard.local/api/tasks ./scripts/load-test.sh` instead (a real DB query per request).

12 🎤 M10 — README, PPT & the Bonus Video

🎤 "*Whatever you build — application code, testing, Git/GitHub, Dockerfiles, CI/CD pipeline, Trivy scan, Terraform, Kubernetes, Helm, and last, observability — you need to create one final presentation and documentation.*"

12.1 Two documents, two readers

	`README.md` (or a `docs/` folder of `.md` files)	The PPT
Reader	an engineer who wants to **run** it	"*any non-technical person*"
Tone	every command, every endpoint, every output	"*Don't add four lines, five lines in each page*" — main points only
Contains	tech stack · how to run · APIs · screenshots of every step	what · why · how · alternatives · 4 proof screenshots
Where	repo root	**repo root** too ("*add the same PPT in the root folder of this project*")

The README order she wants — mirror of her own README's parts:

```
1. Understand the application  (features, APIs, task statuses, priority badges)
2. Build it: frontend + backend + database
3. Run it MANUALLY (no Docker)
4. Dockerfile per microservice
5. docker compose — all three services
6. Kubernetes (Minikube / kind — or EKS/AKS if you have cloud)
7. Terraform files (no apply needed)
8. CI/CD pipeline — and a link to a successful run
```

The PPT, in her words: "*If any non-technical person sees your PPT, he or she should be able to understand what you built.*" Cover:

- the **tech stack**, **what** you built and **why**
- **how** you built it
- **alternatives** — "*what alternatives are there… but you chose only Kubernetes, not any other component*"
- **four proof screenshots:**

#	Screenshot
1	**CI/CD pipeline** — Actions → the run → the green job graph
2	**Main repository** — the code base
3	**Kubernetes deployment** running locally
4	**Docker Compose** — the app up manually + `docker compose ps` in the terminal

🎤 "*Presentation is one of the necessary things for your end term… from your presentation only we will evaluate, and then we will give you the plus 5 points for a better presentation.*"

LOCK THIS IN

The bonus video — +5 points. "*Create a max-to-max 2-minute video in this final presentation… If someone creates a video of their final project, I will give you plus 5 points for this term.*" Later she relaxed the length: "*2 minute or 4 minute, that's up to you — 1 or 2 minutes explaining what you created.*" Not mandatory.

GOOD TO KNOW

On AI. A student said they used AI for the frontend. Her answer — and it's a clear policy: "*Use it for frontend, backend, everything — nobody stopped you. I'm saying it openly: if you're not using AI, use it. But make sure you know what you added … you create it with AI, but you need to understand what AI created.*" (She gave the example of a README written by AI that the student couldn't explain.)

GOOD TO KNOW

Resume-worthy. Asked whether this works for internships: "*This project is perfect for the internship… frontend, backend, Docker Compose, Dockerfile, Terraform, Kubernetes, everything. Even CI/CD's five stages, even DevSecOps concepts. You can add this project inside your resume.*"

---

13 🏋️ The Live Demo — `docker compose up`, a Crash, and the Fix

This was the hands-on part. She'd copied the project into a folder called `DevSecOps Python` on her machine.

13.1 Housekeeping first

```
docker ps                 # kind cluster container still running from last time
kubectl get pods          # → nothing: "Kubernetes is not there"
rm -rf <old-folder>       # cleaned up a stray folder created for "the world data" demo
cd "DevSecOps Python"
```

13.2 The run

```
docker compose up -d --build      # -d = detached, --build = rebuild both images
```

SAID IN CLASS

"It will create my frontend, my backend and my database. If it asks you to allow, then allow it. See — it created your frontend, then backend, and volume also we can see."

```
docker compose ps
```

WATCH IT HAPPEN

Watch it happen — the backend that died on arrival.

```
Frame 1  docker compose up -d --build
           postgres  … Started        backend … Started        frontend … Started
Frame 2  docker compose ps
           frontend   Up    0.0.0.0:3000->80
           postgres   Up    0.0.0.0:5432->5432
           backend    ✗ missing — "something happened with my backend"
Frame 3  docker logs <backend-container>
           … connection to server at "postgres" … failed …
           "It is failed because of connection."
Frame 4  WHY: the backend's CMD runs `alembic upgrade head` the instant it starts.
           Postgres's container was "started" but Postgres itself was still
           initialising its data directory → refused the connection → alembic
           exited non-zero → `&&` never ran uvicorn → container EXITED.
Frame 5  docker compose up -d --build        (again)
           Postgres is warm now → alembic connects → migrates → uvicorn starts
Frame 6  docker compose ps
           backend    Up    0.0.0.0:8000->8000     "See, now backend component is up."
```

CLARIFIED

Clarified, since she said "*once your Postgres container will be up and running, only then the backend container will be created.*" That's the **"started vs ready"** half-truth (it came up in Class 9 §6 too). Plain `depends_on: [postgres]` waits only until the Postgres **container has started** — not until **Postgres is accepting connections**. Her own demo is the proof: the backend started on cue and died anyway.

For her assessment, "`depends_on` controls startup order" is the expected answer. **For your project**, make the wait real with a healthcheck:

```yaml
services:
  postgres:
    image: postgres:16-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U taskboard -d taskboard"]
      interval: 5s
      timeout: 3s
      retries: 10
  backend:
    depends_on:
      postgres:
        condition: service_healthy     # ← now it really waits for "ready"
  frontend:
    depends_on:
      - backend
```

Adding `restart: on-failure` to `backend` is a belt-and-braces extra. Her Helm chart already does the Kubernetes equivalent — Postgres has a `pg_isready` **readiness probe**.

13.3 Testing the frontend — `localhost:3000`

> **SAID IN CLASS**
>
> *"See — the application, the output I wanted. Even this is working — it's not a static application."*

She created a task live:

Field	Value
Title	`DevSecOps project`
Description	`python`
Priority	**HIGH** (started at medium, switched)
Assignee	herself — *"or you can add anyone"*

→ **Create task** → refresh → **Total tasks 1 · To do 1**, the task row with a **HIGH** badge. Clicking ↻ advanced it → refresh → **In progress**.

> **WHY IT WORKS**
>
> **Why "refresh" proves the stack works.** *"Your data, as you refresh, will come from your database to backend to your frontend."* The round trip on every refresh is: browser → Nginx (`/api/tasks`) → FastAPI → SQLAlchemy → Postgres → back. If any one hop were broken, the count would stay at 0 or the red *"Backend unavailable"* banner would appear.

The **Recent activity** panel (*"monitoring dashboard completed, release task moved in progress, onboarding task created, deployment pipeline passed"*) is **hard-coded** — decoration, not data.

13.4 Testing the backend — `localhost:8000`

URL	What she saw
`localhost:8000/`	`{"service":"TaskBoard API","version":"1.0.0","docs":"/docs"}`
`localhost:8000//docs`	❌ *"Not found — why? Because I added two forward slashes. Remove one."*
`localhost:8000/docs`	✅ **Swagger UI** — *"I can test all the APIs for my backend"*
`localhost:8000/health`	`{"status":"UP"}` — *"a string output… the status is up"*
`localhost:8000/metrics`	*"the actual live application metrics"*

Swagger's "Try it out": pick an endpoint → **Try it out** → fill parameters → **Execute** → see the request URL, the response body and the **200** code. *"If your application is working fine, you will see the 200 code."* For `DELETE` you must supply the **task ID**.

> **CLASS MOMENT**
>
> **On Swagger:** *"I know this is a little bit difficult for you — but what you have developed, it will be very easy."* Translation: it feels fiddly on someone else's API; on your own, you'll know what every field means.

HOMEWORK

Screenshots she asked for from this demo (for the README, *not* the PPT):

1. the terminal with `docker compose up -d --build`
2. `docker compose ps`
3. `localhost:3000` with a task created
4. `localhost:8000/docs`
5. `localhost:8000/health`
6. `localhost:8000/metrics`

"*I need this three screenshots or five screenshots of all of the APIs that you are exposing for your backend.*" Then ping in chat when done.

TRY IT YOURSELF

Try it yourself — the same tests from the terminal.

```
curl -s localhost:8000/health
curl -s -X POST localhost:8000/api/tasks -H 'Content-Type: application/json' \
     -d '{"title":"Learn Compose","priority":"HIGH","assignee":"me"}'
curl -s localhost:8000/api/tasks | python3 -m json.tool
curl -s -X PUT localhost:8000/api/tasks/1 -H 'Content-Type: application/json' -d '{"status":"DONE"}'
curl -s localhost:8000/api/tasks/stats
curl -s -o /dev/null -w '%{http_code}\n' -X DELETE localhost:8000/api/tasks/1     # → 204
curl -s localhost:8000/metrics | grep http_requests_total | head
```

Then prove the **volume** works: `docker compose down` → `docker compose up -d` → your tasks are still there. Now `docker compose down -v` → they're gone. That `-v` is the difference.

---

14 🏋️ The Full Practice Lab — Today's Homework, Start to Finish

HOMEWORK

Session-21 homework (instead of a separate homework question): "*You just need to run your Docker Compose, the CI/CD pipeline, the same one that I did… add all the screenshots in a* `README.md` *file*" and submit that README's GitHub link in the **DevOps homework** form.

DO THIS YOURSELF

Do this yourself — the five ways to run TaskBoard.

Stage A — manual, no Docker *("first you need to run it manually")*

```
docker run -d --name pg -p 5432:5432 -e POSTGRES_DB=taskboard \
  -e POSTGRES_USER=taskboard -e POSTGRES_PASSWORD=taskboard postgres:16-alpine   # or a local
Postgres
cd backend
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
export DATABASE_URL='postgresql+psycopg://taskboard:taskboard@localhost:5432/taskboard'
alembic upgrade head
uvicorn app.main:app --reload --port 8000
# new terminal:
cd frontend && npm install && npm run dev        # → http://localhost:5173
```

⚠️ `frontend/vite.config.js` proxies `/api` to `localhost:8080` but the backend is on **8000** — change it to `'http://localhost:8000'` or the dev UI shows "*Backend unavailable*". Also use **Python 3.12**: `psycopg[binary]==3.2.3` has no wheel for very new Pythons (it failed to install on 3.14 when I tried).

Stage B — Docker, one container at a time

```
docker build -t taskboard-backend:local ./backend
docker build -t taskboard-frontend:local ./frontend
docker run --rm -p 8000:8000 \
  -e DATABASE_URL='postgresql+psycopg://taskboard:taskboard@host.docker.internal:5432/taskboard' \
  taskboard-backend:local
```

Stage C — Docker Compose

```
docker compose up -d --build
docker compose ps
docker compose logs backend        # if anything is missing from ps
# test :3000, :8000/docs, :8000/health, :8000/metrics  → screenshots
docker compose down                # keep data
docker compose down -v             # wipe data too
```

Stage D — Kubernetes

```
minikube start
eval $(minikube docker-env)                    # build straight into Minikube's Docker
docker build -t taskboard-backend:local ./backend
docker build -t taskboard-frontend:local ./frontend
kubectl apply -f k8s/namespace.yaml
helm upgrade --install taskboard ./helm/taskboard -n taskboard \
  --set backend.image=taskboard-backend --set backend.tag=local \
  --set frontend.image=taskboard-frontend --set frontend.tag=local \
  --set hpa.enabled=false
kubectl get pods,svc -n taskboard
helm list -n taskboard
```

(Apply the two Service-name fixes from §10 first, or the frontend will CrashLoop.)

Stage E — CI/CD

1. Push the repo to GitHub. 2. Create the Docker Hub PAT (§7.2). 3. Add `DOCKERHUB_TOKEN` (§7.3). 4. Commit `.github/workflows/ci-cd.yml`. 5. `git push` → **Actions** tab → watch five green ticks → screenshot.

15 🔧 Troubleshooting Lab — From Her Repo (Not Demoed)

NOTE

Source note. These two files are in `session21-python/troubleshooting/` and her README's "Part N", but she **didn't run them in this class**. They're here because they're the "troubleshooting" the title promised, and because her exam has a Kubernetes **debugging** question (§16).

Fault 1 — the image that doesn't exist (`broken-image.yaml`): a Deployment pointing at `ghcr.io/example/taskboard-backend:does-not-exist`.

```
kubectl apply -f troubleshooting/broken-image.yaml
kubectl get pods -n taskboard                          # STATUS: ErrImagePull → ImagePullBackOff
kubectl describe pod <pod-name> -n taskboard           # Events: "manifest unknown" / "not found"
kubectl get events -n taskboard --sort-by=.lastTimestamp
```

```
ImagePullBackOff → describe Pod → wrong image/tag → fix the deployment
```

Fix: `kubectl set image deployment/taskboard-broken-image backend=<a real image:tag> -n taskboard` (or edit the YAML and re-apply).

Fault 2 — the Service that selects nothing (`broken-service.yaml`): `selector: {app: label-that-does-not-exist}`.

```
kubectl apply -f troubleshooting/broken-service.yaml
kubectl get svc -n taskboard                  # looks fine — it has a ClusterIP!
kubectl get endpoints broken-service -n taskboard   # ENDPOINTS: <none>   ← the tell
kubectl get pods -n taskboard --show-labels   # compare real labels to the selector
```

LOCK THIS IN

The lesson, in her README's words: "*A Service selects Pods using labels. No matching labels = no endpoints = no traffic.*" A Service never errors for a bad selector — it just quietly routes to nobody. **`kubectl get endpoints`** is the one command that exposes it.

TRY IT YOURSELF

Try it yourself — the CrashLoop exam question. She described the exam's debugging task: "*I will give you a CrashLoop type of image inside your deployment.yaml, and the pod is in CrashLoop; you need to make changes so your pod goes from CrashLoop to Running.*" Rehearse it:

```yaml
# crash.yaml
apiVersion: apps/v1
kind: Deployment
metadata: {name: crashy}
spec:
  replicas: 1
  selector: {matchLabels: {app: crashy}}
  template:
    metadata: {labels: {app: crashy}}
    spec:
      containers:
        - name: c
          image: busybox
          command: ["sh", "-c", "echo starting; exit 1"]    # ← dies immediately
```

```
kubectl apply -f crash.yaml
kubectl get pods                  # Error → CrashLoopBackOff, RESTARTS climbing
kubectl logs deploy/crashy        # "starting"
kubectl logs deploy/crashy --previous
kubectl describe pod -l app=crashy   # Last State: Terminated, Exit Code: 1
# fix: change the command to   ["sh", "-c", "echo starting; sleep 3600"]
kubectl apply -f crash.yaml && kubectl get pods -w     # → Running
```

CrashLoopBackOff = the container starts, then exits, over and over (Kubernetes backs off 10s, 20s, 40s… up to 5 min). The cause is always *inside* the container — read `logs --previous` first.

---

16 🎓 The End-Term Exam Pattern

🎤 *"In the end term you have two types of questions: one is MCQ questions and the second one is lab questions."*

16.1 MCQs

Her worked examples:

Question	Options	Answer
Command to list **only running** containers?	—	`docker ps` (`docker ps -a` = running **+ exited**)
Which command lists all images on your machine?	`docker images` · `docker images -a` · `docker ps` · `docker ps -a`	`docker images`

GOOD TO KNOW

The trap in option 2. `docker images -a` is a real command — it *also* shows intermediate/dangling layers. So it's "all images *plus* plumbing", not the clean answer. Read every MCQ option for these near-misses.

16.2 Lab questions — the shell example she built live

```
mkdir data1
cd data1
mkdir data2
cd data2
echo "hello DevOps folks" > data.txt     # she typed a few variations — "hello students", "DevOps folks"
cat data.txt
cd ../..                                 # back to the starting folder
```

```
<current folder>          ← "you were inside your 06-terraform-vpc directory"
└── data1/
    └── data2/
        └── data.txt      "hello DevOps folks"
```

The question: "*You are inside your main folder. Print the content of `data.txt` from the current folder.*"

Asked	Class answered	Verdict
What's the command?	"`cat` *command*"	⚠️ right tool — she wanted the exact command
The exact command?	`cat data1/data2/data.txt`	✅ "*Correct.*"

> **SAID IN CLASS**
>
> "*This command you just need to run, and it will run some test cases, and then you will complete your lab. This is one of the simplest questions we have for shell scripting.*" — so labs are **auto-graded by test cases** checking what you ran/produced.

16.3 The lab shapes per topic

Topic	What the lab looks like	What you run
Shell	Explore a folder tree, print/list files	`ls`, `cd`, `cat data1/data2/data.txt`
Docker	A `frontend/` folder with Nginx content + a Dockerfile; run it on a **given port** and check `localhost`	`docker build -t web .` → `docker run -d -p <port>:80 web` → `curl localhost:<port>`
Kubernetes	`pod.yml` / `deployment.yml` / `service.yml` given — deploy them	`kubectl apply -f deployment.yml` · `kubectl apply -f service.yml` · `kubectl get pods` — "*once you run these three commands, all test cases will be passed*"
Kubernetes debugging	A Deployment with a CrashLoop image — make it Running (§15)	`logs`, `describe`, fix YAML, `apply`

🎤 Debugging questions are **Kubernetes-only** — "*nothing else… not in Docker.*"

16.4 Practice material — and a promise

- You already have **21 labs**, one per session — "*just practice those labs.*"

- A student asked for lab questions on **Docker networking, volumes and Compose** (the dashboard stopped at Docker lecture 1). She'll write and hand them to the management team "*today*".
- 🎤 **A new practice repo** — "*Let me create one GitHub repo… for 21 sessions, 21 folders, and in each folder two files: one for MCQ questions and one for lab questions to practice.*" Same pattern as the daily assessments and the final exam.

---

17 Deadlines, Homework & the Four Things Left

LOCK THIS IN

"Only four things you need to do from your end."

#	What	Deadline
1	**Final DevOps project** (this class)	**Sunday 11 October 2026**
2	**DevOps homework** — all 21 sessions' homework questions	**Wednesday 7 October, 11:59 PM**
3	**21 assessments / labs** from the management team — per session: 2–3 labs + ~6 MCQs	released by management, "*in these 10 days*"
4	**End-term exam** (MCQ + lab)	"*after 17th October or maybe 20th October*"

How the homework deadline was set — a live negotiation:

CLASS MOMENT

Class moment. She asked how many had already submitted the first 10 homework questions. **156** in the batch, **30** responses. Students pushed for "end of next week" — "*No, no, no. Sorry. We can't keep all the deadlines just before the exam. You will not study anything. You will fail in exam.*" Her arithmetic: you've already had two submission rounds up to Kubernetes; after that came only ~5 sessions (2 CI/CD, 2 Terraform, 1 GitOps) → 5 homework questions in 4 days (Sun–Wed) → **Wednesday 11:59 PM**. "*That is done.*"

How to submit homework (she re-shared her screen to show it):

```
1. Your own "DevOps Heroes" repo → one README.md per session with your answers + screenshots
2. Open that session's README.md on GitHub → copy its URL
3. devops-heros (her repo) → root README.md → "DevOps HomeWork" →
      Section A form  |  Section B form
4. Paste the README link for the matching session → submit
```

She'll update the forms to list every session (they stopped at Kubernetes). A student said some homework prompts were too short — her answer: the detailed tasks are in **each session's** `README.md` in her repo (e.g. for the Kubernetes session: "*for each service you need to create a required YAML file… deployment vs DaemonSet comparison, use cases, pod creation*"); ping her with the session number if anything needs more detail.

GOOD TO KNOW

The confidence line to end on: "*If you are able to do this project, then your end-term project you can complete within hours. Yes — within hours.*"

18 ? Q&A Round-Up

Question (from chat)	Her answer
End-term exam pattern?	**MCQ + lab** (§16)
Any debugging questions?	Yes — **Kubernetes only**: fix a CrashLoop Deployment
What should the PPT cover?	What / why / how / alternatives + 4 screenshots; readable by a non-technical person (§12)
Project deadline?	**Next Sunday** (11 Oct)
Homework deadline?	**Wednesday 11:59 PM** (7 Oct)
Lab questions for Docker networking/volume/ Compose?	She'll create them today and hand them to management
More practice questions?	New repo: 21 folders × (MCQ file + lab file)
Is this the final exam?	No — this is the **final project**; the exam is separate, after ~17–20 Oct
Can I use AI?	Yes, for anything — but you must understand what it produced
Java instead of Python?	**No — Python only**
Is the video compulsory?	No — optional, **+5** points
Good for an internship / resume?	"*Perfect for the internship*"
Where do I submit homework?	Google form (Section A / B) linked in her repo's root README — paste your README's GitHub link
Attendance for a missed lab?	Come to campus tomorrow with **output** (e.g. the VPC from Class 20)

19 Cheat-Sheet

19.1 Docker & Compose

Command	Does
`docker compose up -d --build`	Build both images, start all three services in the background
`docker compose up --build`	Same, but in the foreground (logs streaming) — what the rubric screenshot shows
`docker compose ps`	Status of this project's containers
`docker compose logs backend` / `docker logs <ctr>`	Why did it die?
`docker compose down`	Stop + remove containers, **keep** the volume
`docker compose down -v`	…and **delete** the volume (data gone)
`docker build -t taskboard-backend:local ./backend`	Build one service's image
`docker ps` / `docker ps -a`	Running only / running + exited
`docker images`	All local images
`docker login -u <user>`	Log in with a PAT as the password

19.2 Backend & tests

Command	Does
`python -m venv .venv && source .venv/bin/activate`	Isolated Python env
`pip install -r requirements.txt`	Install deps
`export DATABASE_URL='postgresql+psycopg://…@localhost:5432/taskboard'`	Point the app at a DB
`alembic upgrade head`	Apply all DB migrations
`uvicorn app.main:app --reload --port 8000`	Run FastAPI locally
`pytest -q` / `pytest -v`	Quiet / verbose test run (rubric wants `-v` screenshot)

19.3 Endpoints to screenshot

URL	Expect
`localhost:3000`	TaskBoard UI
`localhost:8000/docs`	Swagger UI
`localhost:8000/health`	`{"status":"UP"}`
`localhost:8000/ready`	`{"status":"READY"}`
`localhost:8000/metrics`	Prometheus text

19.4 CI/CD & secrets

Thing	Where / what
Docker Hub PAT	Profile → Account settings → Personal access tokens → Generate · **30 days** · **Read & Write**
GitHub secret	Repo → Settings → Secrets and variables → Actions → New repository secret
Use it	`${{ secrets.DOCKERHUB_TOKEN }}`
Trigger	`on: push: branches: [main]`
Tag	`${{ github.sha }}` — never `latest`
Trivy gate	`severity: HIGH,CRITICAL` + `exit-code: '1'`
kind in CI	`uses: helm/kind-action@v1.10.0` → `kubectl apply -f k8s/…`

19.5 Terraform

Command	Needs AWS creds?
`terraform fmt -recursive`	No
`terraform init`	No
`terraform validate`	No
`terraform plan`	**Yes**
`terraform apply`	**Yes** — costs money
`aws eks update-kubeconfig --region ap-south-1 --name taskboard-eks`	Yes
`terraform destroy`	Yes — **same day**

19.6 Kubernetes & Helm

Command	Does
`kubectl apply -f k8s/namespace.yaml`	Create the `taskboard` namespace
`helm upgrade --install taskboard ./helm/taskboard -n taskboard --create-namespace`	Install or upgrade the release
`… -f helm/taskboard/values-dev.yaml`	Dev overrides (1 replica, ingress on, HPA off)
`helm list -n taskboard` / `helm history` / `helm rollback taskboard 1`	Releases / revisions / go back
`kubectl get pods,svc,hpa -n taskboard`	The rubric screenshots
`kubectl get endpoints <svc>`	Does the Service actually select any pods?
`kubectl describe pod <p>` / `kubectl logs <p> --previous`	ImagePullBackOff / CrashLoopBackOff detective work
`kubectl get events --sort-by=.lastTimestamp`	What just happened, in order
`minikube addons enable ingress` / `metrics-server`	Needed for Ingress / HPA

19.7 Concepts

Term	One line
Three-tier app	frontend (presentation) → backend (logic) → database (data)
Multi-stage build	Build in a fat image, ship only the output in a slim one
Non-root container	`USER 10001` — a breakout lands as nobody, not root
`depends_on`	Start **order** — not readiness (add a healthcheck)
Liveness vs readiness	"Am I alive?" (`/health`) vs "Can I take traffic?" (`/ready`)
PAT	A scoped, expiring token used instead of a password
SHA tag	Image ↔️ commit traceability; rollback = redeploy an old SHA
kind	Kubernetes-in-Docker; a throwaway cluster inside the CI runner
ServiceMonitor	Tells the Prometheus Operator what to scrape
GitOps	Git is the source of truth — a push *is* the deploy request

20 🗒️ Notes on the Transcript Itself

Instructor. The header's "*Developer: Ritesh prajapati*" is the **Scaler++ Chrome extension's developer**, not the teacher. The instructor is **Nensi Ravaliya** (heard as "*Nancy*" when she mentions the hard-coded greeting).

Title. "*Final Devops Project & Trouble Shooting_2*" — the "Troubleshooting 2" part didn't happen (see the note at the top). Suggested accurate folder name: `Class-21-Final DevOps Project Brief & Exam Prep`.

Quality. `groq, whisper-large-v3-turbo` — good, as with every class since 7. The class was heavily **Hinglish**; Hindi stretches were translated and folded in: - "*batch A ke liye kal hamara ek session hoga*" → for batch A, we have one session tomorrow - "*aisa output mujhe chahiye*" → this is the output I want - "*presentation sahi hona chahiye*" → the presentation must be right - "*Docker Compose application ko up karna*" → bring the Compose app up - "*Late submission mein kya hoga*" → what happens with late submissions - "*frontend ke liye AI use kiya… kisi ne mana nahi kiya… main to samne se bol rahi hoon*" → used AI for the frontend… nobody forbade it… I'm saying it openly - "*Sunday tak rahega*" → stays until Sunday - "*Jo 21 assessment*" → the 21 assessments

Unrecoverable / skipped. The final ~30 seconds drifted into Spanish ASR noise — "*¿Algo más preguntas?… Gracias por ver el video*" ("thanks for watching") — skipped as pure noise. One student exchange about homework detail ("*some of them are quite short… moment of the comedy*") is partly garbled; the gist (prompts felt too short → see each session's README) is in §17.

Garbled-term corrections:

ASR wrote	Meant
"devsickops python", "DevSecOps Python"	the `devsecops-python` repo/folder name she suggested
"devops use", "DevOps Heroes"	her repo `devops-heros`
"3v", "tv", "Trivi", "triviscan"	**Trivy**
"cacd", "CICD", "CIICD"	**CI/CD**
"kine", "Kine cluster"	**kind** cluster
"key it is", "KITS", "ktis", "KTS", "ktest"	the `k8s/` folder
"hem files", "help folder"	**Helm** files / the `helm/` folder
"EKS or EKS" (second one)	**EKS or AKS**
"hashicot"	**HashiCorp**
"stalker.compose", "RockerCompose"	**docker compose**
"talker hub", "docker up"	**Docker Hub**
"post base"	**Postgres**
"dogs"	`/docs`
"medpix", "matrix"	`/metrics`
"swagarwala"	the **Swagger** one
"responsible UI"	**responsive** UI
"clawed ui"	a **Claude**-generated (AI) UI
"githofolder"	`.github/` **folder**
"redmi.md", "unit MD", "trade me.md"	`README.md`
"N-term", "ENTAM", "interim"	**end-term**
"mqdir", "h command"	`mkdir`, `echo`
"self scripting", "size script"	**shell scripting**
"githops"	**GitOps**

Facts verified against files, not the audio: - Pipeline contents, Dockerfiles, Compose, Helm chart, Terraform and tests are quoted from `session21-python/` (your copy is byte-identical to her repo's — `diff -rq` showed no differences). - The Docker-Hub/apt-Trivy/kind pipeline she *described* matches her `session-17-devsecops/demo/.github/workflows/devsecops.yml`, not the `session21-python` workflow — see §7.5. - Bugs flagged in §4, §5, §6, §9, §10 and §11 were **reproduced**: `pytest` on a clean checkout (`no such table: tasks`), `terraform validate` (`Invalid single-argument block definition`), and `helm template` (Service/Ingress name and port mismatch). - The `kube-prometheus-stack` install commands in §11 and the CrashLoop rehearsal in §15 are **filled in** as standard practice — she named the tools and the exam task but didn't type these commands in class.

Your working folder. `session21-python/` is an unmodified copy of her project (no `.git`, no `test.db`, no build output — the `__pycache__/` folders came with the copy, they're in her repo too — so nothing was run from it yet), and `z/` is empty.